Computer Arithmetic and Round-Off Error

Alexandra A. Gomes

February 4, 2026

Significant Digits: Decimal Rounding Representation

After rounding to t digits:

$$fl(x) = \pm d_0.d_1 \ldots d_{t-1} \times 10^n$$

The **rounding unit/machine precision/machine epsilon** is

$$\eta = \frac{1}{2} 10^{1-t}.$$

$t-1$ is the **number of significant digits** in $fl(x)$, which means that $fl(x)$ represents all the numbers in the real interval

$$[fl(x) - \eta \times 10^n, fl(x) + \eta \times 10^n[.$$

Floating Point Arithmetic

$x = 5/7 = 0.\overline{714286}$ and $y = 0.714251$; use 5-digit chopping arithmetic:

$$\begin{aligned}
|(x-y) \quad - \quad (x \ominus y)| &= |(x-y) - fl\,(fl\,(x) - fl\,(y))| \\
&= \left|\left(\frac{5}{7} - 0.714251\right) - fl\left(7.1428 \times 10^{-1} - 7.1425 \times 10^{-1}\right)\right| \\
&= |3.47143\ldots \times 10^{-5} - 3 \times 10^{-5}| \\
&= 4.7143 \times 10^{-6} \\
&\leq 10^{-5}
\end{aligned}$$

$$\frac{|(x-y) - (x \ominus y)|}{|x-y|} = \frac{4.7143 \times 10^{-6}}{3.4714 \times 10^{-5}} = 0.136 \geq 10^{-4}$$

the relative error is larger than its upper bound and therefore we need to take measures, such as using guard digits or avoiding subtractions.

Floating Point Arithmetic

Let z be a real number and $z + \delta$ be its floating point representation. Here, the error δ comes either from representation or previous calculations. Then,

$$fl\left(\frac{fl(z)}{fl(\varepsilon)}\right) = (z + \delta) \times 10^n.$$

with $\varepsilon = 10^{-n}, n > 0$.

Loss of Significance: Cancellation of Significant Digits in Subtraction

Consider $x = 5/7 = 0.\overline{714286}$ and $y = 0.714251$ from our last example:

$$fl(x) = 7.1428 \times 10^{-1} \qquad fl(y) = 7.1425 \times 10^{-1},$$

accurate to 5 digits and calculate $x \ominus y$ with 5-digit chopping arithmetic:

$$x \ominus y = fl\,(0.00003) = 3 \times 10^{-5}$$

which cannot be accurate to 5 digits. This is **loss of significance**.

Floating Point Arithmetic

$x = 5/7 = 0.\overline{714286}$ and $y = 0.714251$; use 5-digit rounding arithmetic:

$$\frac{|(x-y)-(x \ominus y)|}{|x-y|} = \frac{5.2857 \times 10^{-6}}{3.4714 \times 10^{-5}} = 0.155 \leq \frac{1}{2}10^{1-t} = \frac{1}{2}10^{0}$$

conclusion:

1. x has an infinite number of significant digits;
2. $y = 7.14251 \times 10^{-1}$ has 5 significant digits;
3. the approximation to $x - y$ has only 1 significant digit.

Roundoff Error Accumulation

1. $x \approx y$: look for cancellation error; $x - y$ has a large relative error;
2. $|x| \gg |y|$: $x + y$ has a large absolute error;
3. $|y| \ll 1$: both the absolute and the relative error of x/y may be large;

Loss of Significance: Rationalization

Idea: use rationalization to avoid subtraction

Example: Given $f(x) = \sqrt{x^2+1} - 1$, calculate $f(0.011) \approx 6 \times 10^{-5}$ with 4-digit rounding arithmetic.

- $f(0) = \sqrt{1} - 1$
- $f(x) = \sqrt{x^2+1} - 1 = \left(\sqrt{x^2+1} - 1\right) \frac{\left(\sqrt{x^2+1}+1\right)}{\left(\sqrt{x^2+1}+1\right)} = \frac{x^2}{\sqrt{x^2+1}+1}$

Loss of Significance: Taylor series

Idea: use Taylor polynomials to avoid subtraction

Example: $f(x) = x - \sin x$

- $f\left(10^{-4}\right) = 10^{-4} - \sin 10^{-4}$
- $f(x) = x - \left(x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots\right) = \frac{x^3}{3!} - \frac{x^5}{5!} + \frac{x^7}{7!} + \ldots$

Reducing Round-Off Error: Nesting Polynomials

Idea: reduce round-off errors by decreasing the number of calculations

$$
\begin{aligned}
p(x) &= 1 + 2x + 3x^2 + 4x^3 + 5x^4 \\
&= 1 + 2x + 3x^2 + x^3(4 + 5x) \\
&= 1 + 2x + x^2(3 + x(4 + 5x)) \\
&= 1 + x(2 + x(3 + x(4 + 5x)))
\end{aligned}
$$

Binary Floating-Point Representation

$x \neq 0 \in \mathbb{R}$ has a binary representation as

$$x = \pm (1.b_1 b_2 \ldots b_{t-1} b_t b_{t+1} \ldots)_2 \times 2^n$$

$b_i \in \{0, 1\}, i = 1, 2, 3, \ldots$
$n \in \mathbb{Z}$

The floating-point representation of $x \neq 0$ with t digits is

$$fl(x) = \pm \left(1.b_1 b_2 \ldots \tilde{b}_t\right)_2 \times 2^n$$

example: what is the decimal representation of

$$x = -(1.1)_2 \times 2^1?$$

Binary Floating-Point Representation

$$fl(x) = \pm \left(1.b_1 b_2 \ldots \tilde{b}_t\right)_2 \times 2^n$$

requires

- t bits for the mantissa;
- 1 bit for $\pm$;
- a fixed number of bits for the exponent: $n_L \leq n \leq n_U$,

with **rounding unit**

$$\eta = \frac{1}{2} 2^{-t}.$$

IEEE Standard Floating-Point Representation

double precision or long precision: 64-bit word

$$fl(x) = \pm \left(1.f_1 f_2 \ldots \tilde{f}_t\right)_2 \times 2^n$$

$$b_1 b_2 \ldots b_{63} b_{64}$$

$$\underbrace{(-1)^{b_1}}_{\text{1 bit for } \pm} \times \underbrace{2^{(b_2 b_3 \ldots b_{12})_2}}_{\text{11 bits for } n} \times 2^{-1023} \times \underbrace{(1.b_{13} \ldots b_{64})_2}_{\text{52 bits for mantissa}}$$

$$0 < (b_2 b_3 \ldots b_{12})_2 < (11111111111)_2 = 2047$$

$$n_L = -1022 \leq n \leq 1023 = n_U$$

$$\eta = \frac{1}{2} 2^{-t} = 2^{-t-1} = 2^{-53} \approx 1.1 \times 10^{-16}$$

Round-Off Error, Machine Precision

What is an upper bound for the round-off error in a 64-bit machine with rounding arithmetic? η ?

$$x = \pm (1.f_1 f_2 \ldots)_2 \times 2^n$$

$$x^- = \pm (1.f_1 f_2 \ldots f_{52})_2 \times 2^n$$

$$x^+ = \pm (1.f_1 f_2 \ldots f_{52})_2 \times 2^n + 2^{-52} \times 2^n$$

If x^- is closest to x in the real line, then

$$|x - x^-| \leq \frac{1}{2}|x^+ - x^-| = 2^{n-53} \Rightarrow \frac{|x - x^-|}{|x|} \leq 2^{-53} = \eta$$

2η is the **round-off error chopping arithmetic**

Binary Floating-Point Representation: Example

0 10000000011
1011100100010000000000000000000000000000000000000000
$= 27.56640625$

the next smallest number is 0 10000000011
1011100100001111111111111111111111111111111111111111
$= 27.56640624999999822364 3\ldots$

the next largest number is 0 10000000011
1011100100010000000000000000000000000000000000000001
$= 27.56640625000001776357\ldots$

which means that the floating-point representation is not continuous!

Machine with $t = 2$, $-2 \leq n \leq 3$

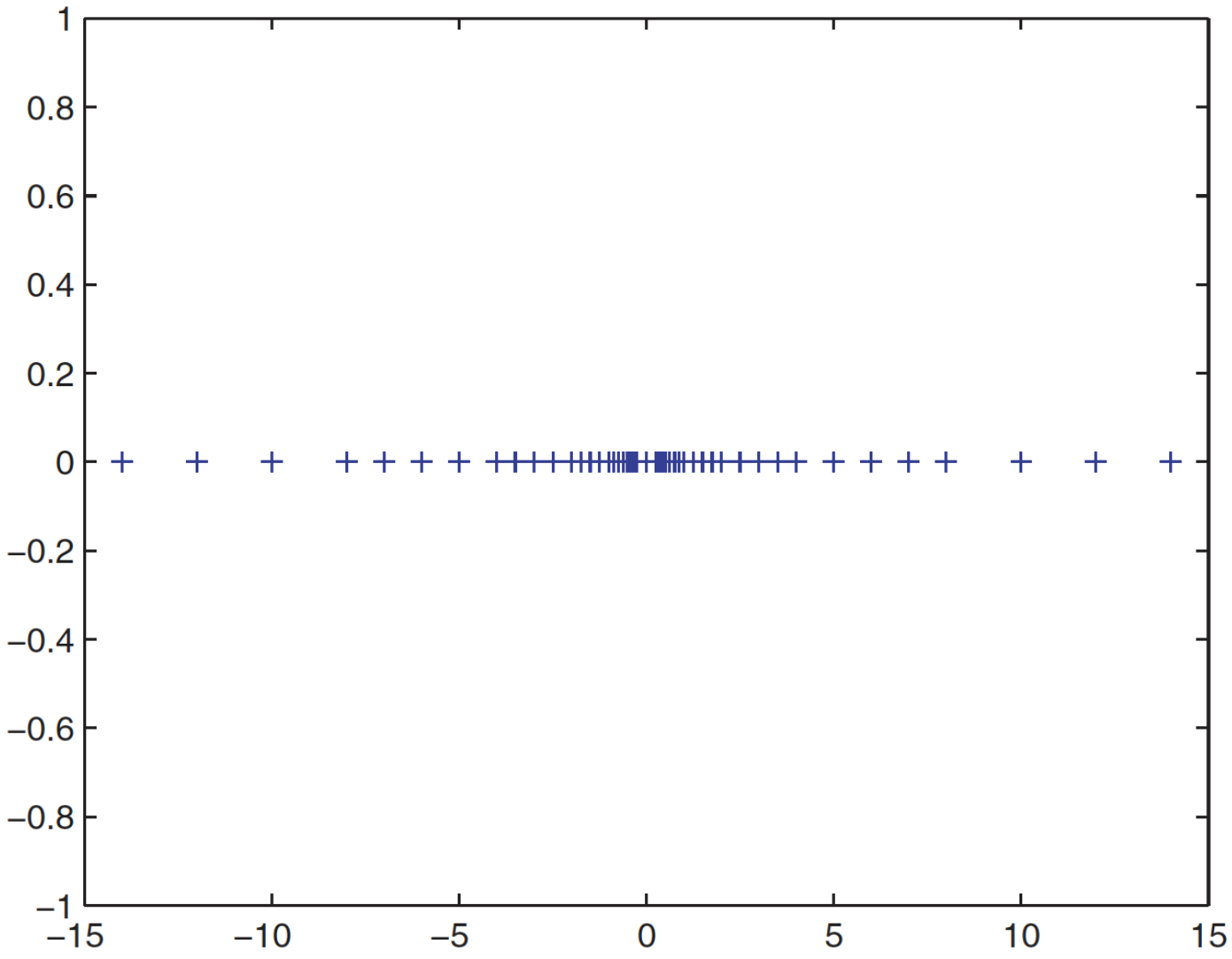

Figure 2.3. *Picture of the floating point system described in Example* 2.8.

Binary Floating-Point Representation

$$\underbrace{(-1)^{b_1}}_{\text{1 bit for } \pm} \times \underbrace{2^{(b_2 b_3 \ldots b_{12})_2}}_{\text{11 bits for } n} \times 2^{-1023} \times \underbrace{(1.b_{13} \ldots b_{64})_2}_{\text{52 bits for mantissa}}$$

- the smallest positive machine number (all b's = 0):

$$2^{-1022} \times (1+0) \approx 2.2251 \times 10^{-308};$$

- any number between 0 and 2.2251×10^{-308} is an **underflow**;
- the largest positive machine number:

$$2^{1023} \times \left(2 - 2^{-52}\right) \approx 1.7977 \times 10^{308};$$

- any number larger than 1.7977×10^{308} is an **overflow**;